

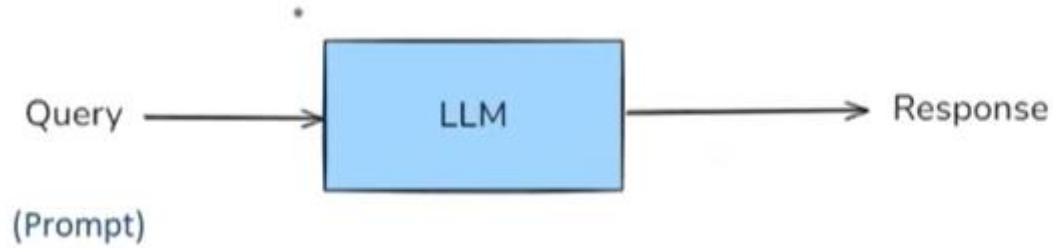
Retrieval-Augmented Generation

RAG Components Overview

RAG consists of 4 main components:

1. Document Loader - Loads data from various sources
2. Text Splitter - Breaks documents into chunks
3. Vector Store - Stores embeddings for retrieval
4. Retrievers - Fetches relevant documents

What is RAG?



Limitations of LLM

- 1) Can not access private data*
- 2) Cut-off knowledge*
- 3) Hallucination*

Solution : Fine-Tuning

Updating model weights by training on a specific dataset. Requires computational resources but deeply adapts the model to specialized domains.

Types of Fine Tuning

- 1) *Supervised*
- 2) *Unsupervised*
- 3) *RLHF*

1. Collect data	A few hundred – few hundred-thousand carefully curated examples (prompts → desired outputs).
2. Choose a method	Full-parameter FT, LoRA/QLoRA, or parameter-efficient adapters.
3. Train for a few epochs	You keep the base weights frozen or partially frozen and update only a small subset (LoRA) or all weights (full FT).
4. Evaluate & safety-test	Measure exact-match, factuality, and hallucination rate against held-out data; red-team for safety.

Solution : Fine-Tuning

Fine tuning solve the Limitations of LLM

- 1) *Can not access private data*
- 2) *Cut-off knowledge*
- 3) *Hallucination*

Problems with Fine tuning

- 1) *To train LLM is computationally expensive*
- 2) *Technical expertise required*
- 3) *Frequent data update*

In-Context Learning (ICL)

A core capability of LLMs (GPT-3/4, Claude, Llama) where the model learns tasks by seeing examples in the prompt without updating weights.

Key Advantage: Fast adaptation without retraining

When to use Fine Tuning: Domain-specific tasks beyond ICL capability

When not to use Fine Tuning : General tasks (ICL is faster and cheaper)

In-Context Learning (ICL)

Below are examples of texts labeled with their sentiment.
Use these examples to determine the sentiment of the final text.

Text: I love this phone. It's so smooth. → Positive
Text: This app crashes a lot. → Negative
Text: The camera is amazing! → Positive
|
Text: I hate the battery life. →

Label the named entities in the sentences (Person, Location, Organization)

Sentence: Elon Musk founded SpaceX.

Entities: [Elon Musk → Person], [SpaceX → Organization]

Sentence: The Eiffel Tower is in Paris.

Entities: [Eiffel Tower → Location], [Paris → Location]

Sentence: Sundar Pichai leads Google.

Entities:|

An **emergent property** is a behaviour or ability that suddenly appears in a system when it reaches a certain scale or complexity—even though it was not explicitly programmed or expected from the individual components.

<https://arxiv.org/pdf/2005.14165>

Info

- Instead of just *example tasks*, retrieve **background information**, facts, documents, product manuals, etc.
- Inject that into the prompt to **augment the model's knowledge**

```

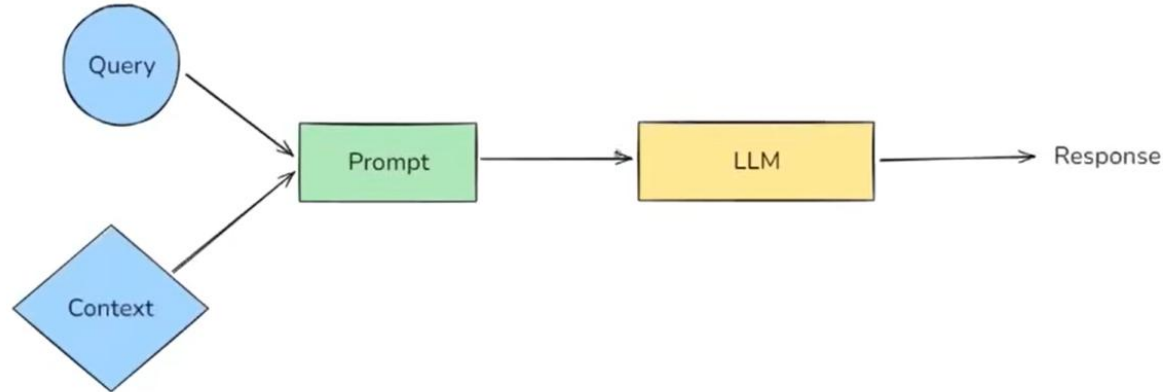
"""You are a helpful assistant.
Answer the question ONLY from the provided context.
If the context is insufficient, just say you don't know.

{context}
Question: {question}"""

```

What is RAG

RAG is a way to make a language model (like ChatGPT) smarter by giving it extra information at the time you ask your question.



- Indexing
- Retrieval
- Augmentation
- Generation

Understanding RAG - 4 Steps

Indexing

Prepare knowledge base for efficient searching

Retrieval

Find most relevant information from index

Augmentation

Combine retrieved context with user query

Generation

LLM generates response using augmented prompt

Phase 1: Indexing

Process of preparing knowledge base for efficient search. Four sub-steps:

1. Document Ingestion - Load source knowledge into memory

🔥 Examples:

- PDF reports, Word documents
- YouTube transcripts, blog pages
- GitHub repos, internal wikis
- SQL records, scraped webpages

🔧 Tools:

- LangChain loaders (PyPDFLoader, YoutubeLoader, WebBaseLoader, GitLoader, etc.)



Phase 1: Indexing

Process of preparing knowledge base for efficient search. Four sub-steps:

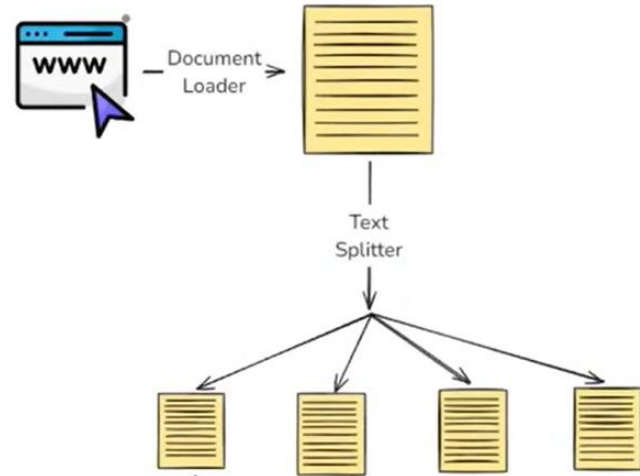
2. Text Chunking - Break documents into semantically meaningful chunks

Why chunk?

- LLMs have context limits (e.g., 4K–32K tokens)
- Smaller chunks are more focused → better semantic search

Tools

- `RecursiveCharacterTextSplitter`, `MarkdownHeaderTextSplitter`, `SemanticChunker`



Phase 1: Indexing

Process of preparing knowledge base for efficient search. Four sub-steps:

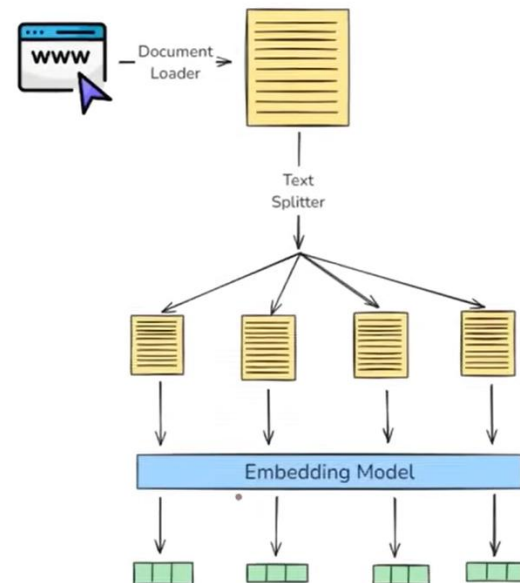
3. Embedding Generation : Convert each chunk into a **dense vector** (embedding) that capture its meaning,

🔍 Why embeddings?

- Similar ideas land close together in vector space
- Allows fast, fuzzy semantic search

🔧 Tools:

- `OpenAIEmbeddings`, `SentenceTransformerEmbeddings`, `InstructorEmbeddings`, etc.



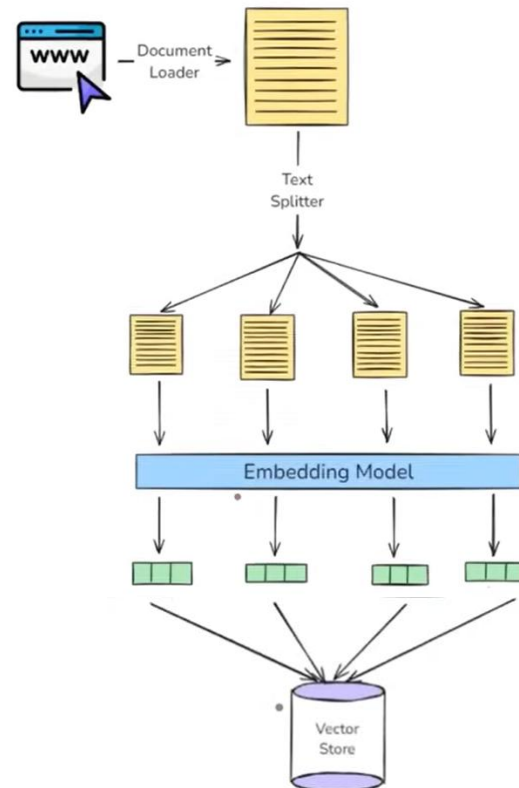
Phase 1: Indexing

Process of preparing knowledge base for efficient search. Four sub-steps:

4. Storage in a Vector Store : Store the vectors along with the original chunk te

Vector DB options:

- Local: FAISS, Chroma
- Cloud: Pinecone, Weaviate, Milvus, Qdrant

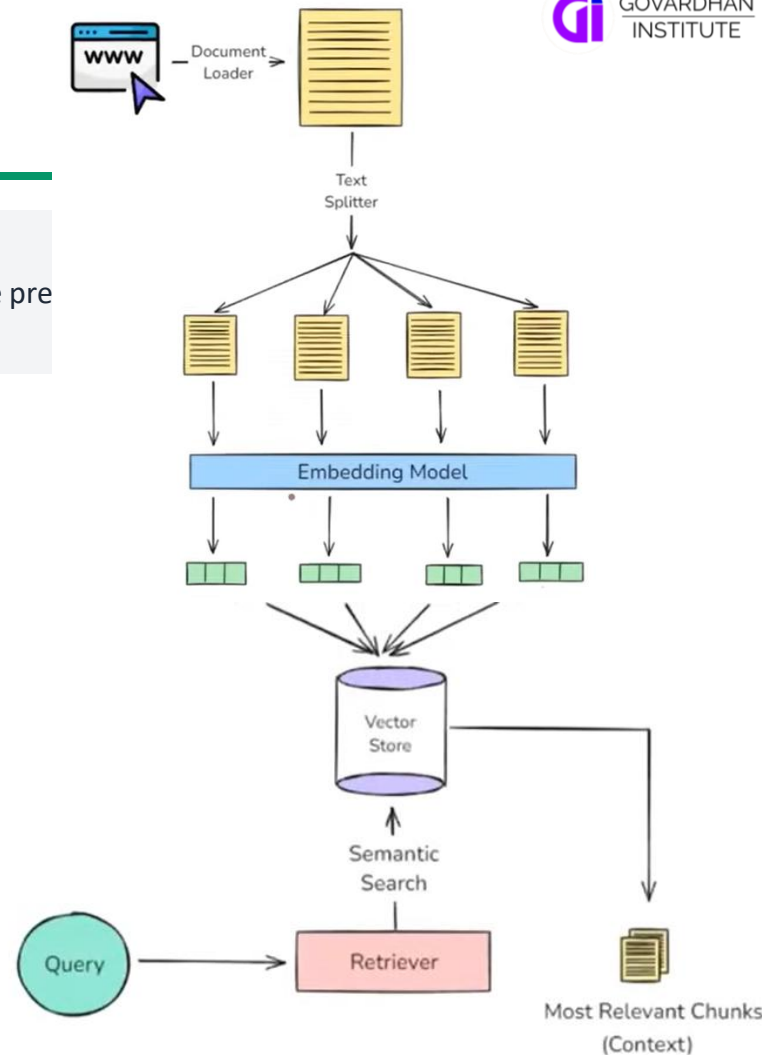


Phase 2: Retrieval

Real-time process of finding the most relevant pieces of information from the pre

Core Question: Which 3-5 chunks best answer this query?

Uses semantic similarity search with vector embeddings

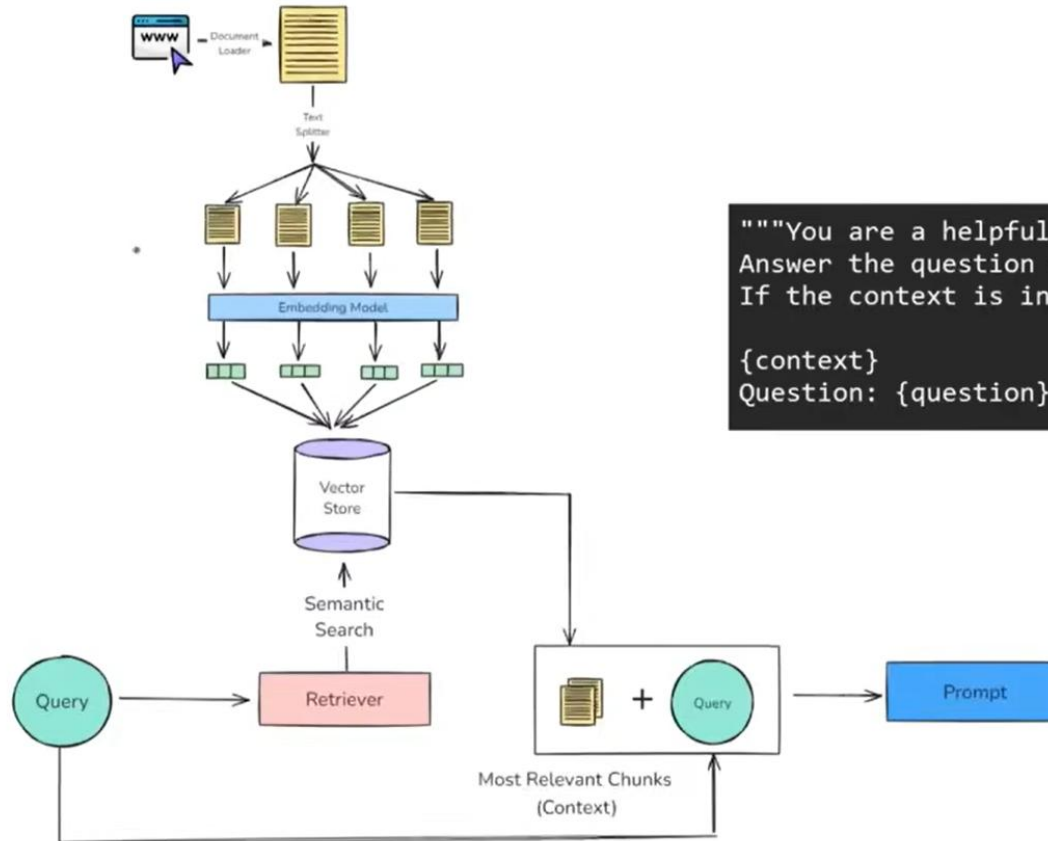


Phase 3: Augmentation

Step where retrieved documents are combined with user's query to form a new, enriched prompt for the LLM.

Format: System Prompt + Retrieved Context + User Question → Enhanced Prompt

Phase 3: Augmentation



```

"""You are a helpful assistant.
Answer the question ONLY from the provided context.
If the context is insufficient, just say you don't know

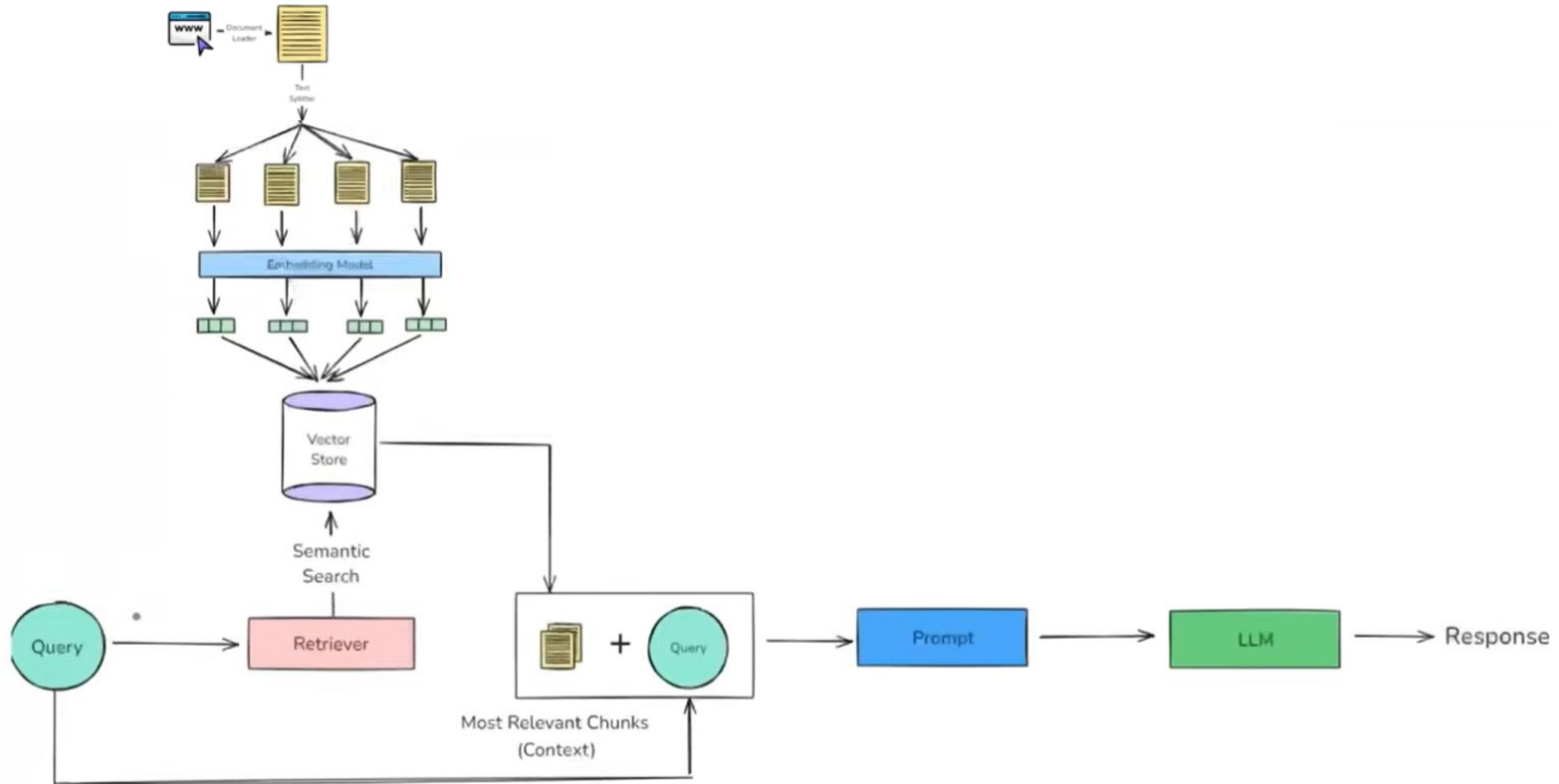
{context}
Question: {question}"""
    
```

Phase 4: Generation

Final step where LLM uses user's query and retrieved, augmented context to generate a comprehensive response.

Result: Factually grounded, informed response backed by actual documents

Phase 4: Generation



RAG Summary & Benefits

1. RAG bridges LLMs with up-to-date external knowledge
2. Four phases: Indexing, Retrieval, Augmentation, Generation
3. In-Context Learning for quick adaptation without retraining
4. Fine-Tuning for domain-specific deep customization
5. Emergent properties appear unexpectedly at model scale
6. Solves: outdated data, hallucinations, factual errors
7. Complete flow: Index docs → Query → Retrieve → Augment → Generate